

Chapter 10. Chain Rule, Jacobians, and Automatic Differentiation

Compositions, sensitivity, computational graphs, and backpropagation

The chain rule is the rule for differentiating a function built from other functions. In single-variable calculus, if $y = f(u)$ and $u = g(t)$, then

$$\frac{dy}{dt} = \frac{dy}{du} \frac{du}{dt}.$$

In multivariable calculus, the same idea becomes more powerful because each intermediate quantity may depend on several variables, and each output may have several components. The derivative is no longer just one number. It may be a gradient vector, a Jacobian matrix, or a linear transformation.

This chapter is one of the main bridges from classical multivariable calculus to modern computation. The chain rule explains how input changes propagate through a model, why Jacobian matrices describe local sensitivity, how coordinate transformations are differentiated, and why backpropagation in neural networks is repeated use of the chain rule.

Core principle

The multivariable chain rule says that derivatives propagate through compositions by multiplying the correct local linear maps.

Teaching-material connection

This chapter develops partial derivatives, gradients, tangent-plane approximation, directional derivatives, and implicit differentiation into a systematic treatment of chain rules and Jacobians. The same notation later supports change of variables, optimization, statistics, machine learning, and vector calculus.

Learning goals

After studying this chapter, you should be able to:

1. recognize compositions of one-variable, two-variable, and vector-valued functions;
2. apply the chain rule when $z = f(x, y)$ and $x = x(t)$, $y = y(t)$;
3. apply the chain rule when $z = f(x, y)$ and $x = x(s, t)$, $y = y(s, t)$;
4. compute Jacobian matrices for maps $F : \mathbb{R}^n \rightarrow \mathbb{R}^m$;
5. write the multivariable chain rule in matrix form;
6. interpret the Jacobian as the local linear map of a nonlinear function;
7. compute and verify Jacobians numerically using finite differences;
8. understand computational graphs as organized compositions;
9. explain reverse-mode automatic differentiation and backpropagation at a conceptual

level.

1 Why composition matters

A function of several variables often appears indirectly. For example, suppose

$$z = f(x, y) = x^2y + \sin y,$$

but x and y themselves depend on time:

$$x = t^2, \quad y = e^t.$$

Then z is ultimately a function of t :

$$z(t) = f(x(t), y(t)) = t^4 e^t + \sin(e^t).$$

We could substitute first and then differentiate. But in applications, substitution may be inconvenient or impossible. The chain rule computes the derivative by following the dependencies.

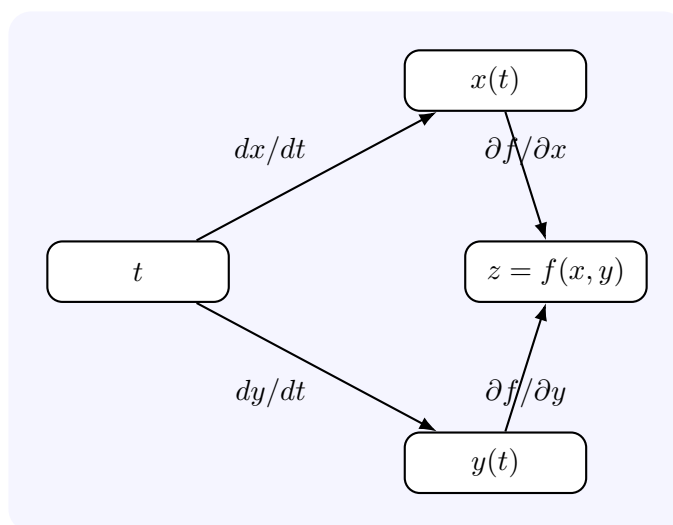


Figure 1: A composed function passes information from input variables through intermediate variables to the final output.

The input t affects x and y . Then x and y affect z . Therefore a small change in t changes z through two routes:

$$t \rightarrow x \rightarrow z, \quad t \rightarrow y \rightarrow z.$$

The chain rule adds the contributions from all routes.

2 Chain rule along a curve

Suppose $z = f(x, y)$ is differentiable, and $x = x(t)$ and $y = y(t)$ are differentiable functions of t . Then

$$z(t) = f(x(t), y(t))$$

has derivative

$$\frac{dz}{dt} = \frac{\partial f}{\partial x} \frac{dx}{dt} + \frac{\partial f}{\partial y} \frac{dy}{dt}.$$

All partial derivatives are evaluated at $(x(t), y(t))$.

Equivalently, using the gradient,

$$\frac{dz}{dt} = \nabla f(x(t), y(t)) \cdot \frac{d}{dt} \langle x(t), y(t) \rangle.$$

Thus

Chain rule along a curve

$$\frac{d}{dt} f(r(t)) = \nabla f(r(t)) \cdot r'(t).$$

The rate of change of a scalar field along a curve is the gradient dotted with the velocity vector.

Worked example 10.0: Chain rule along a curve

Let

$$f(x, y) = x^2 y + \sin y, \quad x(t) = t^2, \quad y(t) = e^t.$$

Find dz/dt at $t = 0$.

The partial derivatives are

$$f_x(x, y) = 2xy, \quad f_y(x, y) = x^2 + \cos y.$$

At $t = 0$,

$$x(0) = 0, \quad y(0) = 1,$$

and

$$x'(0) = 0, \quad y'(0) = 1.$$

Therefore

$$\left. \frac{dz}{dt} \right|_{t=0} = f_x(0, 1)x'(0) + f_y(0, 1)y'(0) = 0 \cdot 0 + \cos(1) \cdot 1 = \cos(1).$$

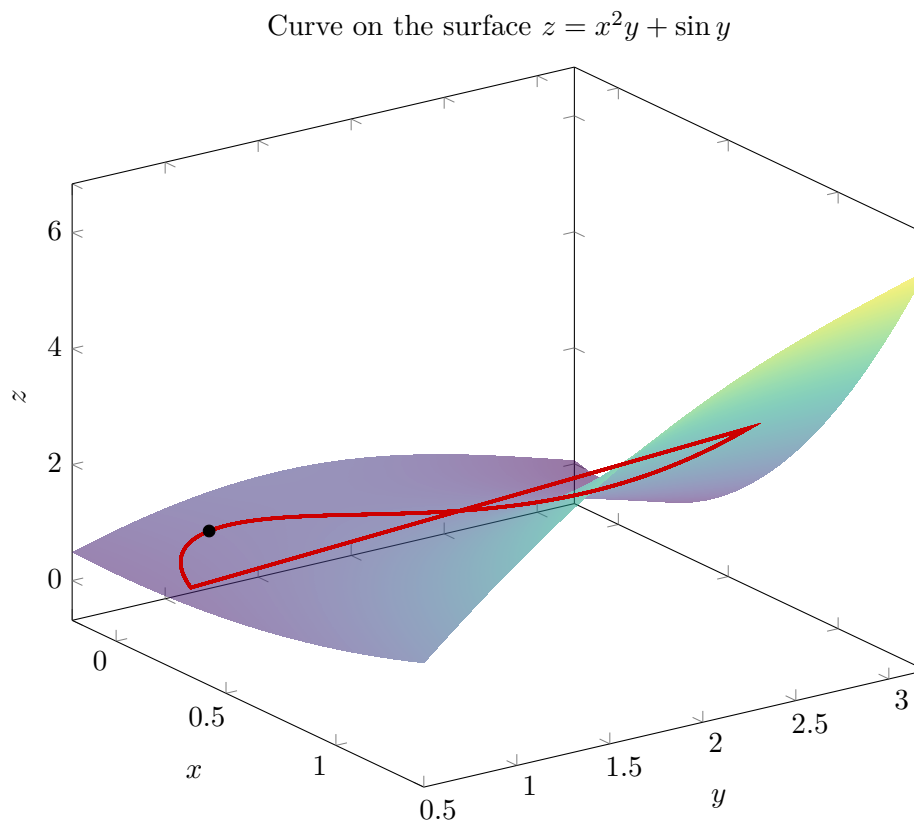


Figure 2: The chain rule measures the rate at which a surface changes along a parameterized curve in the domain.

3 The tree form of the chain rule

Now suppose

$$z = f(x, y), \quad x = x(s, t), \quad y = y(s, t).$$

Then z is a function of two variables s and t :

$$z(s, t) = f(x(s, t), y(s, t)).$$

The partial derivatives of z are

$$\frac{\partial z}{\partial s} = \frac{\partial f}{\partial x} \frac{\partial x}{\partial s} + \frac{\partial f}{\partial y} \frac{\partial y}{\partial s},$$

and

$$\frac{\partial z}{\partial t} = \frac{\partial f}{\partial x} \frac{\partial x}{\partial t} + \frac{\partial f}{\partial y} \frac{\partial y}{\partial t}.$$

A useful memory device is

$$(s, t) \longrightarrow (x, y) \longrightarrow z.$$

To differentiate z with respect to one input variable, follow every path from that variable to z , multiply derivatives along each path, and add the path contributions.

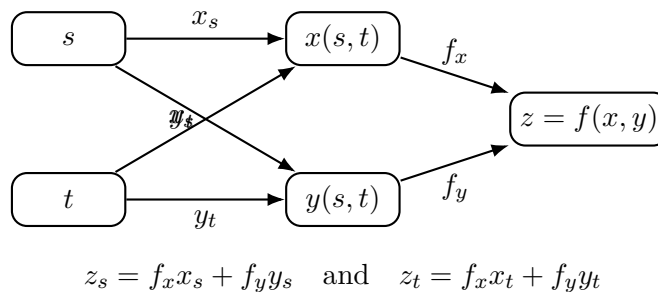


Figure 3: The tree form of the chain rule adds the derivative contribution from every dependency path.

Worked example 10.1: Two-parameter chain rule

Let

$$f(x, y) = x^2 + y^3, \quad x = s + t, \quad y = st.$$

Then $f_x = 2x$ and $f_y = 3y^2$. Also

$$x_s = 1, \quad x_t = 1, \quad y_s = t, \quad y_t = s.$$

Therefore

$$z_s = 2x \cdot 1 + 3y^2 \cdot t = 2x + 3ty^2,$$

and

$$z_t = 2x \cdot 1 + 3y^2 \cdot s = 2x + 3sy^2.$$

Substituting $x = s + t$ and $y = st$ gives

$$z_s = 2(s + t) + 3t(s^2 t^2), \quad z_t = 2(s + t) + 3s(s^2 t^2).$$

4 Jacobian matrices

Let

$$F : \mathbb{R}^n \rightarrow \mathbb{R}^m,$$

where

$$F(x_1, \dots, x_n) = \begin{bmatrix} F_1(x_1, \dots, x_n) \\ \vdots \\ F_m(x_1, \dots, x_n) \end{bmatrix}.$$

The *Jacobian matrix* of F is

$$J_F(x) = \begin{bmatrix} \frac{\partial F_1}{\partial x_1} & \frac{\partial F_1}{\partial x_2} & \cdots & \frac{\partial F_1}{\partial x_n} \\ \frac{\partial F_2}{\partial x_1} & \frac{\partial F_2}{\partial x_2} & \cdots & \frac{\partial F_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial F_m}{\partial x_1} & \frac{\partial F_m}{\partial x_2} & \cdots & \frac{\partial F_m}{\partial x_n} \end{bmatrix}.$$

The entry in row i , column j tells how the i th output changes when the j th input changes.

Dimensions matter

If $F : \mathbb{R}^n \rightarrow \mathbb{R}^m$, then J_F is an $m \times n$ matrix. It maps small input displacements $\Delta x \in \mathbb{R}^n$ to small output displacements $\Delta F \in \mathbb{R}^m$:

$$\Delta F \approx J_F(x)\Delta x.$$

Worked example 10.2: A Jacobian matrix

Let

$$F(x, y) = \begin{bmatrix} x^2y \\ e^{x+y} \\ \sin(xy) \end{bmatrix}.$$

Then $F : \mathbb{R}^2 \rightarrow \mathbb{R}^3$. Its Jacobian is a 3×2 matrix:

$$J_F(x, y) = \begin{bmatrix} 2xy & x^2 \\ e^{x+y} & e^{x+y} \\ y \cos(xy) & x \cos(xy) \end{bmatrix}.$$

At $(1, 0)$,

$$J_F(1, 0) = \begin{bmatrix} 0 & 1 \\ e & e \\ 0 & 1 \end{bmatrix}.$$

Thus if (x, y) moves by a small vector $\Delta x = \langle h, k \rangle$, then

$$\Delta F \approx \begin{bmatrix} 0 & 1 \\ e & e \\ 0 & 1 \end{bmatrix} \begin{bmatrix} h \\ k \end{bmatrix} = \begin{bmatrix} k \\ e(h+k) \\ k \end{bmatrix}.$$

5 The Jacobian as a local linear map

For a scalar function $f : \mathbb{R}^n \rightarrow \mathbb{R}$, the derivative at a point is encoded by the gradient. For a vector-valued function $F : \mathbb{R}^n \rightarrow \mathbb{R}^m$, the derivative is encoded by the Jacobian. Near a point a ,

$$F(a + h) \approx F(a) + J_F(a)h.$$

This means that the nonlinear function behaves like an affine linear map near a .

Worked example 10.3: Local effect of a nonlinear map

Let

$$F(x, y) = \begin{bmatrix} x + y \\ x^2 - y \end{bmatrix}.$$

Then

$$J_F(x, y) = \begin{bmatrix} 1 & 1 \\ 2x & -1 \end{bmatrix}.$$

At $a = (1, 1)$,

$$J_F(1, 1) = \begin{bmatrix} 1 & 1 \\ 2 & -1 \end{bmatrix}.$$

Therefore a small square around $(1, 1)$ in the input plane is approximately mapped to a parallelogram in the output plane.

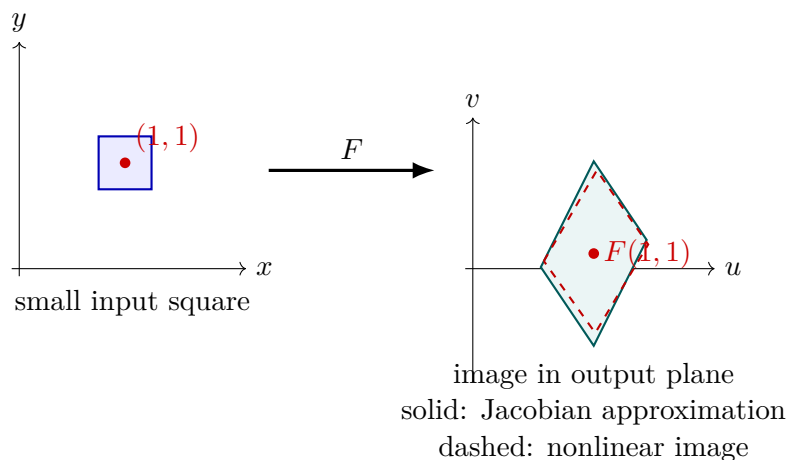


Figure 4: Near a point, a nonlinear map is approximated by its Jacobian linear map.

6 Matrix form of the chain rule

Suppose

$$F : \mathbb{R}^n \rightarrow \mathbb{R}^m, \quad G : \mathbb{R}^m \rightarrow \mathbb{R}^p.$$

Then the composition $G \circ F : \mathbb{R}^n \rightarrow \mathbb{R}^p$ has Jacobian

Matrix chain rule

$$J_{G \circ F}(x) = J_G(F(x)) J_F(x).$$

The order is important: first apply F , then apply G , but the Jacobian matrices multiply as outer derivative times inner derivative.

This is the multivariable version of $(g \circ f)'(x) = g'(f(x))f'(x)$.

Worked example 10.4: Matrix chain rule

Let

$$F(s, t) = \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} s + t \\ st \end{bmatrix}, \quad G(x, y) = x^2 + y^3.$$

The Jacobian of F is

$$J_F(s, t) = \begin{bmatrix} 1 & 1 \\ t & s \end{bmatrix}.$$

The Jacobian of G is the row vector

$$J_G(x, y) = \begin{bmatrix} 2x & 3y^2 \end{bmatrix}.$$

Therefore

$$J_{G \circ F}(s, t) = \begin{bmatrix} 2x & 3y^2 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ t & s \end{bmatrix} = \begin{bmatrix} 2x + 3ty^2 & 2x + 3sy^2 \end{bmatrix}.$$

This matches the tree form of the chain rule.

7 Gradients through linear maps

A very common situation in data science and machine learning is

$$L(w) = \phi(Aw + b),$$

where $w \in \mathbb{R}^n$ is a parameter vector, A is a matrix, b is a vector, and ϕ is a scalar loss function. Let

$$u = Aw + b.$$

Then the chain rule gives

Gradient through a linear map

$$\nabla_w L(w) = A^T \nabla_u \phi(u), \quad u = Aw + b.$$

The value moves forward through $u = Aw + b$. The gradient moves backward through A^T .

Worked example 10.5: Least-squares gradient

Let

$$L(w) = \frac{1}{2} \|Xw - y\|^2,$$

where X is a data matrix, w is a parameter vector, and y is an observed response vector. Define $r = Xw - y$. Then

$$L = \frac{1}{2} r^T r.$$

The gradient with respect to r is $\nabla_r L = r$. Since $r = Xw - y$, the chain rule gives

$$\nabla_w L = X^T (Xw - y).$$

This is one of the most important gradients in applied mathematics, statistics, and machine learning.

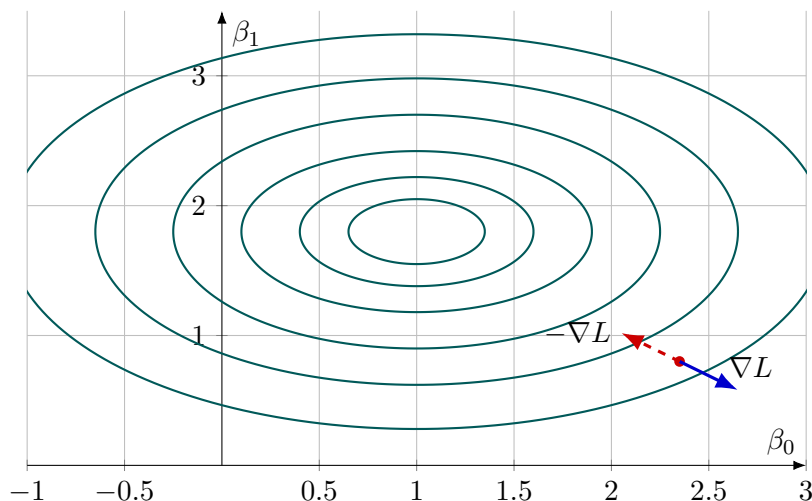


Figure 5: The least-squares gradient points toward increasing loss, while gradient descent moves in the opposite direction.

8 Numerical Jacobians by finite differences

In applications, we may have a function implemented as code but not have a closed formula. We can approximate the Jacobian using finite differences. For $F : \mathbb{R}^n \rightarrow \mathbb{R}^m$, the j th column of the Jacobian can be approximated by

$$\frac{F(x + he_j) - F(x - he_j)}{2h},$$

where e_j is the j th standard basis vector.

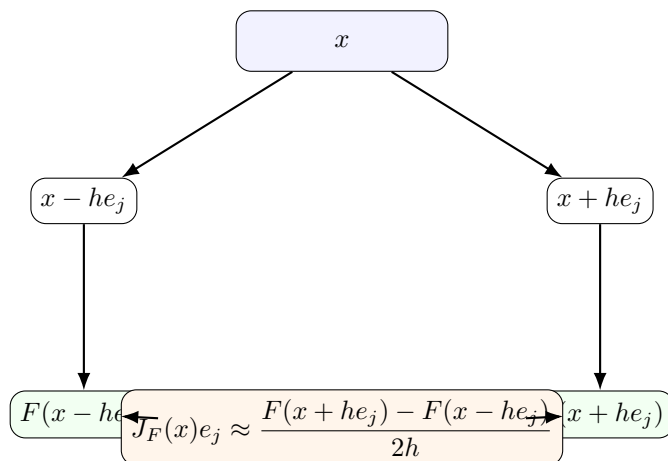


Figure 6: A centered finite difference estimates one column of the Jacobian at a time.

Finite differences are useful, but they can be slow or unstable for very large models. Automatic differentiation gives a more efficient and accurate way to compute derivatives of functions defined by code.

9 Computational graphs

A *computational graph* breaks a complicated function into elementary operations. Each node stores a value. Each edge represents dependence.

Consider

$$f(x, y) = (xy + \sin x)^2.$$

We can compute it step by step:

$$a = xy, \quad b = \sin x, \quad c = a + b, \quad f = c^2.$$

The chain rule can be applied locally at every step.

Worked example 10.6: Forward sensitivity

For $f(x, y) = (xy + \sin x)^2$, set

$$a = xy, \quad b = \sin x, \quad c = a + b, \quad f = c^2.$$

Then

$$\frac{\partial f}{\partial c} = 2c, \quad \frac{\partial c}{\partial a} = 1, \quad \frac{\partial c}{\partial b} = 1,$$

and

$$\frac{\partial a}{\partial x} = y, \quad \frac{\partial a}{\partial y} = x, \quad \frac{\partial b}{\partial x} = \cos x.$$

Therefore

$$\frac{\partial f}{\partial x} = 2c(y + \cos x), \quad \frac{\partial f}{\partial y} = 2cx.$$

Since $c = xy + \sin x$,

$$\nabla f(x, y) = \langle 2(xy + \sin x)(y + \cos x), 2x(xy + \sin x) \rangle.$$

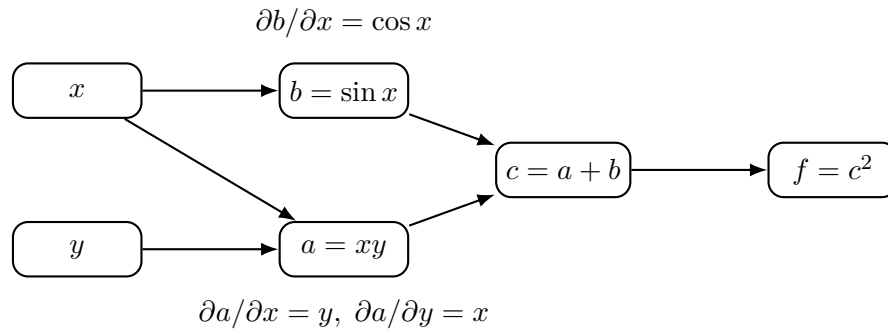


Figure 7: A computational graph decomposes a function into elementary operations where local derivatives are easy to compute.

10 Reverse-mode automatic differentiation

Automatic differentiation is not symbolic differentiation and not finite differences. It computes exact derivatives of the operations performed by a program, up to floating-point rounding.

There are two main modes:

- *Forward mode* propagates derivatives from inputs to outputs.
- *Reverse mode* propagates derivatives from output back to inputs.

Reverse mode is especially efficient when there is one scalar output and many inputs, which is exactly the situation for many loss functions:

$$L(\theta_1, \dots, \theta_n) : \mathbb{R}^n \rightarrow \mathbb{R}.$$

Backpropagation is reverse-mode automatic differentiation applied to neural networks.

Worked example 10.7: Reverse accumulation

Again let

$$f = (xy + \sin x)^2.$$

At each node, reverse mode stores an *adjoint*, meaning the derivative of the final output with respect to that node. Start with

$$\bar{f} = \frac{\partial f}{\partial f} = 1.$$

Since $f = c^2$,

$$\bar{c} = \bar{f} \frac{\partial f}{\partial c} = 2c.$$

Since $c = a + b$,

$$\bar{a} = \bar{c}, \quad \bar{b} = \bar{c}.$$

Since $a = xy$ and $b = \sin x$,

$$\bar{x} = \bar{a}y + \bar{b} \cos x, \quad \bar{y} = \bar{a}x.$$

This gives the same gradient as before, but the organization is different: compute values forward, then gradients backward.

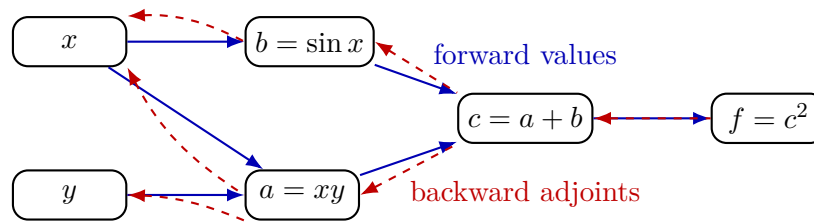


Figure 8: Reverse mode first evaluates the graph forward, then propagates adjoints backward from the scalar output.

11 A tiny neural-network example

Consider a very small model with one input x , one weight w , one bias b , and sigmoid activation

$$\hat{y} = \sigma(wx + b), \quad \sigma(u) = \frac{1}{1 + e^{-u}}.$$

For a target value y , use the squared loss

$$L(w, b) = \frac{1}{2}(\hat{y} - y)^2.$$

The computation is

$$u = wx + b, \quad \hat{y} = \sigma(u), \quad L = \frac{1}{2}(\hat{y} - y)^2.$$

The chain rule gives

$$\frac{\partial L}{\partial w} = (\hat{y} - y)\sigma'(u)x, \quad \frac{\partial L}{\partial b} = (\hat{y} - y)\sigma'(u),$$

where

$$\sigma'(u) = \sigma(u)(1 - \sigma(u)).$$

This is backpropagation in its smallest possible form.

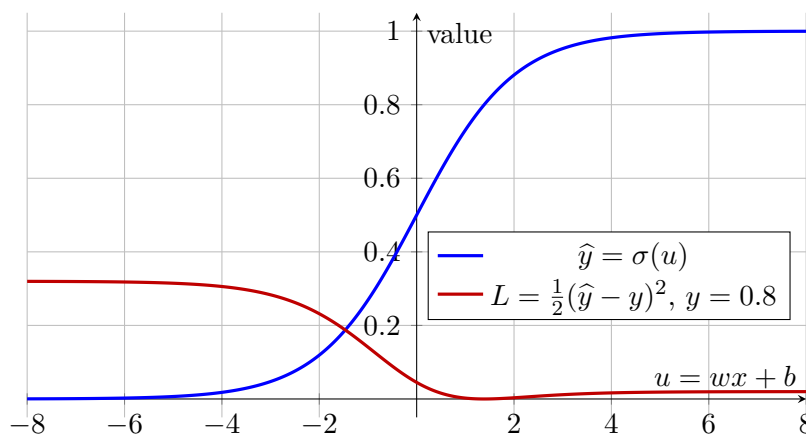


Figure 9: A one-neuron model is a composition of an affine map, a nonlinear activation, and a loss function.

12 Chain rule and coordinate transformations

Coordinate transformations are also compositions. For polar coordinates,

$$x = r \cos \theta, \quad y = r \sin \theta.$$

If $z = f(x, y)$, then

$$z(r, \theta) = f(r \cos \theta, r \sin \theta).$$

The chain rule gives

$$\frac{\partial z}{\partial r} = f_x \cos \theta + f_y \sin \theta,$$

and

$$\frac{\partial z}{\partial \theta} = f_x(-r \sin \theta) + f_y(r \cos \theta).$$

The Jacobian matrix of the polar coordinate map

$$\Phi(r, \theta) = \begin{bmatrix} r \cos \theta \\ r \sin \theta \end{bmatrix}$$

is

$$J_{\Phi}(r, \theta) = \begin{bmatrix} \cos \theta & -r \sin \theta \\ \sin \theta & r \cos \theta \end{bmatrix}.$$

Its determinant is

$$\det J_{\Phi}(r, \theta) = r.$$

This determinant explains why the area element becomes

$$dA = r \, dr \, d\theta.$$

Thus the chain rule for derivatives and the change-of-variables rule for integrals are closely connected.

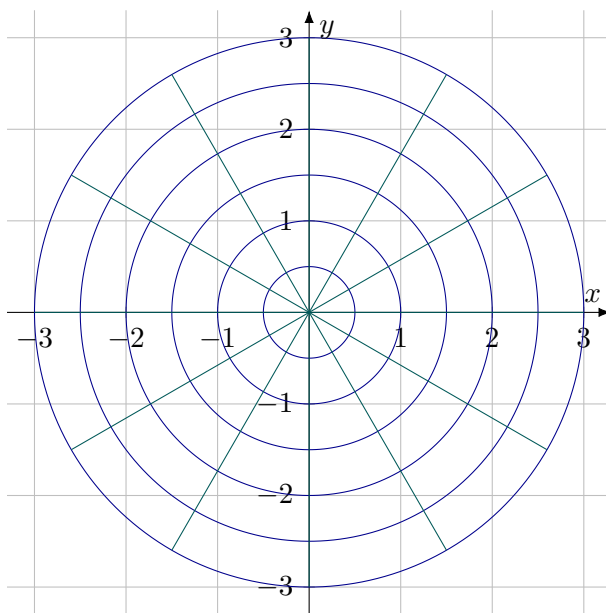


Figure 10: The polar coordinate map bends a rectangular grid in parameter space into radial and circular coordinate curves.

13 Implicit differentiation as a chain rule

Suppose $z = z(x, y)$ is defined implicitly by

$$F(x, y, z) = 0.$$

Then $F(x, y, z(x, y))$ is identically zero. Differentiating with respect to x gives

$$F_x + F_z \frac{\partial z}{\partial x} = 0.$$

Therefore, if $F_z \neq 0$,

$$\boxed{\frac{\partial z}{\partial x} = -\frac{F_x}{F_z}}.$$

Similarly,

$$\boxed{\frac{\partial z}{\partial y} = -\frac{F_y}{F_z}}.$$

This is the same chain rule idea, but now one variable is hidden inside an equation.

Worked example 10.8: Implicit partial derivatives

Suppose $z = f(x, y)$ is defined by

$$x^3 + y^3 + z^3 + 2xyz = 3.$$

Let

$$F(x, y, z) = x^3 + y^3 + z^3 + 2xyz - 3.$$

Then

$$F_x = 3x^2 + 2yz, \quad F_y = 3y^2 + 2xz, \quad F_z = 3z^2 + 2xy.$$

Thus

$$\frac{\partial z}{\partial x} = -\frac{3x^2 + 2yz}{3z^2 + 2xy}, \quad \frac{\partial z}{\partial y} = -\frac{3y^2 + 2xz}{3z^2 + 2xy}.$$

14 Common mistakes

Mistake 1: Forgetting all paths

If $z = f(x, y)$ and both x and y depend on t , then dz/dt has two contributions:

$$\frac{dz}{dt} = f_x x'(t) + f_y y'(t).$$

Do not keep only one path.

Mistake 2: Multiplying Jacobians in the wrong order

For $G \circ F$, the derivative is

$$J_{G \circ F}(x) = J_G(F(x))J_F(x),$$

not $J_F(x)J_G(F(x))$. Check matrix dimensions.

Mistake 3: Confusing gradient and Jacobian

For a scalar-valued function $f : \mathbb{R}^n \rightarrow \mathbb{R}$, the Jacobian is a $1 \times n$ row vector, while the gradient is usually written as an $n \times 1$ column vector or as a vector in $\mathbb{R}^n$. The information is the same, but the orientation matters in matrix formulas.

Mistake 4: Treating automatic differentiation as numerical differentiation

Finite differences estimate derivatives by evaluating nearby points. Automatic differentiation applies the chain rule exactly to the operations in a program. These are different methods.

15 Summary of core formulas

Chain rule along a curve

If $z = f(x(t), y(t))$, then

$$\frac{dz}{dt} = f_x \frac{dx}{dt} + f_y \frac{dy}{dt}, \quad \frac{d}{dt}f(r(t)) = \nabla f(r(t)) \cdot r'(t).$$

Chain rule with two parameters

If $z = f(x(s, t), y(s, t))$, then

$$z_s = f_x x_s + f_y y_s, \quad z_t = f_x x_t + f_y y_t.$$

Jacobian matrix and linear approximation

For $F : \mathbb{R}^n \rightarrow \mathbb{R}^m$,

$$J_F(x) = \begin{bmatrix} \frac{\partial F_i}{\partial x_j} \end{bmatrix}, \quad F(a + h) \approx F(a) + J_F(a)h.$$

Matrix chain rule

$$J_{G \circ F}(x) = J_G(F(x))J_F(x).$$

Gradient through a linear map

If $L(w) = \phi(Aw + b)$, then

$$\nabla_w L = A^T \nabla_u \phi(u), \quad u = Aw + b.$$

Implicit differentiation

If $F(x, y, z(x, y)) = 0$, then

$$z_x = -\frac{F_x}{F_z}, \quad z_y = -\frac{F_y}{F_z}.$$

16 Practice problems

Conceptual problems

1. Explain why the chain rule is a rule about composition.
2. Explain why the chain rule adds contributions from different paths in a dependency graph.
3. What is the dimension of the Jacobian matrix for a map $F : \mathbb{R}^5 \rightarrow \mathbb{R}^3$?

4. Explain the difference between a gradient and a Jacobian.
5. Why does reverse-mode automatic differentiation work well for scalar loss functions with many parameters?
6. Explain why A^T appears when backpropagating a gradient through $u = Aw + b$.

Computational and algebraic problems

7. Let $z = x^2 + xy + y^2$, where $x = t^2$ and $y = \sin t$. Find dz/dt .
8. Let $z = e^{xy}$, where $x = s + t$ and $y = s - t$. Find z_s and z_t .
9. Let $F(x, y) = \langle x^2 + y, xy, \sin(x + y) \rangle$. Compute $J_F(x, y)$.
10. Let $F(x, y) = \langle x + y, x - y \rangle$ and $G(u, v) = u^2 + v^2$. Compute $J_{G \circ F}$ using matrix multiplication.
11. Let $F(x, y, z) = x^2 + y^2 + z^2 - 1 = 0$ define $z = z(x, y)$ near a point with $z \neq 0$. Find z_x and z_y .
12. Let $L(w) = \frac{1}{2} \|Aw - y\|^2$. Derive $\nabla_w L = A^T(Aw - y)$.
13. For $f(x, y) = (x^2 + y)^3$, draw a computational graph and compute ∇f by the chain rule.
14. For $\hat{y} = \sigma(w_1 x_1 + w_2 x_2 + b)$ and $L = \frac{1}{2}(\hat{y} - y)^2$, compute $\partial L / \partial w_1$, $\partial L / \partial w_2$, and $\partial L / \partial b$.
15. Use finite differences to approximate the Jacobian of $F(x, y) = \langle e^{xy}, x^2 - y^2 \rangle$ at $(1, 2)$.

Challenge problems

16. Let $F : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ be $F(r, \theta) = \langle r \cos \theta, r \sin \theta \rangle$. Compute J_F and explain geometrically why $|\det J_F| = r$.
17. Let $F(x) = Ax$ and $G(u) = \frac{1}{2} \|u - y\|^2$. Use the matrix chain rule to derive the least-squares gradient.
18. Suppose $H = G \circ F$ where $F : \mathbb{R}^n \rightarrow \mathbb{R}^m$ and $G : \mathbb{R}^m \rightarrow \mathbb{R}$. Show that $\nabla H(x) = J_F(x)^T \nabla G(F(x))$ when gradients are written as column vectors.
19. Write a short paragraph explaining how backpropagation is related to the chain rule.
20. Consider $f(x, y, z) = \sin(xy) + z^2$ and a curve $r(t) = \langle t, e^t, \cos t \rangle$. Compute $d(f \circ r)/dt$ at $t = 0$.

17 Python lab and interactive page

- Notebook lab: `Chapter10_Chain_Rule_Jacobians_and_Automatic_Differentiation.ipynb`
- Interactive HTML page: `../htmls/chapter-10.html`

18 Chapter summary

The chain rule is the calculus of composition. In its simplest form, it differentiates $f(g(t))$. In multivariable calculus, it differentiates functions passing through curves, surfaces, coordinate transformations, and computational graphs. The Jacobian matrix organizes these derivative calculations as linear maps. The matrix chain rule

$$J_{G \circ F}(x) = J_G(F(x))J_F(x)$$

is the central formula of the chapter.

For modern applied mathematics, statistics, and machine learning, the most important lesson is that derivatives can be propagated through models. Least-squares gradients, logistic-regression gradients, neural-network backpropagation, coordinate transformations, and sensitivity analysis are all expressions of the same idea: compute local derivatives at each step and combine them by the chain rule.